Buổi 8

Đa năng hóa toán tử (phần 2) và sao chép sâu

Học phần: Lập trình hướng đối tượng

Nội dung bài học

1. Đa năng hóa một số toán tử đặc biệt

1.1 Toán tử chỉ số []
1.2 Toán tử gọi hàm
1.3 Toán tử chuyển đổi kiểu
1.4 Đa năng hóa toán tử new và delete

2. Toán tử chèn dòng << và trích dòng >>

2.1 Đa năng hóa << và >>
2.2 Ví dụ lớp Date với <<, >> và ++

3. Sao chép sâu: copy constructor và operator=

3.1 Vấn đề sao chép nông với lớp có con trỏ
3.2 Copy constructor và operator= cho lớp String
3.3 Lớp String hoàn chỉnh
3.4 Kiểm thử sao chép sâu – theo dõi các hàm được gọi
3.5 Phân tích kết quả kiểm thử
3.6 Áp dụng Quy tắc ba: lớp Mang

4. Tổng kết và thực hành

4.1 Tóm tắt buổi 8
4.2 Bài thực hành buổi 8 – Lớp String có cấp phát động, kiểm thử sao chép sâu
4.3 Câu hỏi và bài tập

1.1 Toán tử chỉ số []

- `a[i]` được dịch thành `a.operator[](i)`; **bắt buộc** là hàm thành viên. Thường trả về **tham chiếu** để dùng được ở **cả hai vế** của phép gán.

```
class Mang
{  public:
      Mang(int n);
      ~Mang();
      int &operator[](int i)                // đọc + ghi
      {  if (i < 0 || i >= N)
         {  cerr << "Loi chi so " << i << endl;
            exit(1);
         }
         return A[i];
      }
      int operator[](int i) const           // cho đối tượng
hằng
      {  return A[i];  }
      int Size() const { return N; }
   private:
      int *A;  int N;
};
Mang m(5);
m[2] = 9;              // m.operator[](2) = 9  (ghi)
cout << m[2];          // 9                    (đọc)
m[7] = 1;              // "Loi chi so 7" và dừng
```

- **Lợi ích so với mảng C:**
  - **Kiểm tra chỉ số** – phát hiện ngay lỗi truy cập ngoài mảng (mảng C ghi đè bộ nhớ âm thầm).
  - Dùng cú pháp quen thuộc `m[i]` thay vì `m.Get(i)`, `m.Set(i, x)`.
- **Vì sao trả về `int &`?** Nếu trả `int` (giá trị) thì `m[2] = 9` chỉ gán vào **bản sao tạm** ⇒ lỗi biên dịch "lvalue required".
- **Hai phiên bản** (const và không const) – đa năng hóa theo const: đối tượng hằng gọi bản const (chỉ đọc), đối tượng thường gọi bản trả tham chiếu.

Lưu ý

`operator[]` chỉ nhận **một** tham số; muốn truy cập ma trận `a[i][j]` thì `[]` thứ nhất trả về một "dòng" (con trỏ hoặc đối tượng dòng), hoặc dùng toán tử `()` ở trang sau.

1.2 Toán tử gọi hàm ()

- `a(x, y, …)` được dịch thành `a.operator()(x, y, …)`; bắt buộc là thành viên; nhận **số tham số tùy ý** (toán tử duy nhất được có tham số mặc định).

Ví dụ 8.1 – Truy cập phần tử ma trận

```
class MaTran
{  public:
      MaTran(int m, int n)
      {  M = m;  N = n;  A = new double[m * n];
         for (int i = 0; i < m * n; i++) A[i] = 0;
      }
      ~MaTran() { delete[] A; }
      double &operator()(int i, int j)
      {  return A[i * N + j];  }     // lưu theo dòng
   private:
      double *A;  int M, N;
};
MaTran a(2, 3);
a(1, 2) = 7.5;
cout << a(1, 2) << " " << a(0, 0);   // 7.5 0
```

Ví dụ 8.2 – Chuỗi con theo giáo trình: `s(Index, SubLen)`

```
String s("happy birthday");
String t = s(6, 8);       // "birthday"
```

- **Đối tượng hàm (functor):** đối tượng có `operator()` dùng được **như một hàm**, nhưng mang theo **trạng thái**:

```
class LonHon
{  public:
      LonHon(int n) { Nguong = n; }
      bool operator()(int x) const { return x > Nguong; }
   private:
      int Nguong;
};
LonHon lonHon5(5);
cout << lonHon5(7) << lonHon5(3);   // 1 0
```

- Dùng nhiều trong thư viện chuẩn: tiêu chí sắp xếp, lọc dữ liệu (`sort`, `count_if`).

1.3 Toán tử chuyển đổi kiểu

- **Chiều vào** (kiểu khác ⇒ lớp): **constructor một tham số** (buổi 7). **Chiều ra** (lớp ⇒ kiểu khác): **toán tử chuyển kiểu** `operator KieuDich() const` – **không ghi kiểu trả về**, không có tham số.

```
class PhanSo
{  public:
      PhanSo(int t = 0, int m = 1) { Tu = t; Mau = m; }
      operator double() const          // PhanSo ⇒ double
      {  return double(Tu) / Mau;  }
   private:
      int Tu, Mau;
};
PhanSo p(3, 4);
double x = p;                  // 0.75   (ngầm)
cout << p + 1.5;               // 2.25   (p ⇒ double)
cout << sqrt(double(p));       // 0.866025 (tường minh)
```

- Toán tử chuyển kiểu cũng dùng cho `if (doiTuong)` bằng `operator bool()` – chính là cách `while (cin >> x)` hoạt động (istream có toán tử chuyển sang bool).

Cẩn thận: nhập nhằng khi có cả hai chiều chuyển kiểu

Lớp có **cả** `PhanSo(int)` **và** `operator double()` cùng `operator+(PhanSo, PhanSo)`: biểu thức `p + 1` có thể hiểu là `p + PhanSo(1)` hoặc `double(p) + 1` ⇒ trình biên dịch báo **ambiguous**. Giải pháp: khai báo `explicit` cho một chiều (C++11 cho phép `explicit operator double()`), hoặc viết hàm `ToDouble()` thay cho toán tử.

Hướng chuyển	Cách cài đặt	Ví dụ
int ⇒ PhanSo	Constructor `PhanSo(int t, int m = 1)`	`PhanSo a = 5;`
const char* ⇒ String	Constructor `String(const char *s)`	`String s = "abc";`
PhanSo ⇒ double	`operator double() const`	`double x = p;`
String ⇒ const char*	`operator const char*() const`	`strlen(s)`

1.4 Đa năng hóa toán tử new và delete

- Có thể định nghĩa lại `new`/`delete` **toàn cục** (ảnh hưởng mọi cấp phát) hoặc **cho một lớp** (chỉ đối tượng lớp đó) – để theo dõi bộ nhớ, dùng vùng nhớ riêng (memory pool), tăng tốc cấp phát nhiều đối tượng nhỏ.

```
class Diem
{  public:
      Diem(int a = 0, int b = 0) { x = a; y = b; }
      void *operator new(size_t size)
      {  SoLan++;
         cout << "  [new Diem: " << size << " byte]\n";
         return malloc(size);
      }
      void operator delete(void *p)
      {  cout << "  [delete Diem]\n";
         free(p);
      }
      static int SoLan;
   private:
      int x, y;
};
int Diem::SoLan = 0;
```

```
Diem *d = new Diem(1, 2);  delete d;
Diem *e = new Diem;        delete e;
cout << "So lan new: " << Diem::SoLan;
```

```
  [new Diem: 8 byte]
  [delete Diem]
  [new Diem: 8 byte]
  [delete Diem]
So lan new: 2
```

- **Quy tắc:**
  - `operator new` nhận `size_t` (kích thước cần cấp phát), trả về `void *`; `operator delete` nhận `void *`, trả về void.
  - Hai hàm này **tự động là static** (gọi khi đối tượng chưa tồn tại / đã hủy).
  - Chỉ thay việc **cấp phát bộ nhớ**; constructor/destructor **vẫn** được gọi như thường sau `new` / trước `delete`.
  - Cặp mảng: `operator new[]`, `operator delete[]`.
- Toàn cục: `void *operator new(size_t size) { … }` đặt ngoài mọi lớp.

2.1 Đa năng hóa << và >>

- `cout << a` được dịch thành `operator<<(cout, a)`: toán hạng trái là **ostream** ⇒ **không thể** là hàm thành viên của lớp ta (phải là thành viên của ostream) ⇒ cài đặt bằng hàm **friend** (cần truy cập private).

```
class Date
{  friend ostream &operator<<(ostream &os, const Date &d);
   friend istream &operator>>(istream &is, Date &d);
   public:
      Date(int d = 1, int m = 1, int y = 2000);
      Date &operator++();            // sang ngày kế tiếp
   private:
      int Day, Month, Year;
};
ostream &operator<<(ostream &os, const Date &d)
{  os << setfill('0') << setw(2) << d.Day << "/"
      << setw(2) << d.Month << "/" << d.Year << setfill(' ');
   return os;                        // để viết nối tiếp
}
istream &operator>>(istream &is, Date &d)
{  char c;                           // bỏ dấu '/'
   is >> d.Day >> c >> d.Month >> c >> d.Year;
   return is;
}
```

- **Tham số và kiểu trả về:**

Hàm	Tham số 1	Tham số 2	Trả về
`<<`	`ostream &`	`const Date &` (chỉ đọc)	`ostream &`
`>>`	`istream &`	`Date &` (để ghi vào)	`istream &`

- **Vì sao trả về tham chiếu dòng?** Để viết **nối tiếp**: `cout << d1 << " -> " << d2;` được dịch thành `operator<<(operator<<(operator<<(cout, d1), " -> "), d2)`.
- Dùng tham số `os`, **không** dùng trực tiếp `cout` trong hàm ⇒ hàm còn dùng được cho **tập tin** (`ofstream`) và chuỗi (`ostringstream`) – buổi 11.

2.2 Ví dụ lớp Date với <<, >> và ++

```
Date &Date::operator++()          // sang ngày kế tiếp
{  static const int SoNgay[13] =
      {0,31,28,31,30,31,30,31,31,30,31,30,31};
   int MaxD = SoNgay[Month];
   if (Month == 2 && ((Year % 4 == 0 && Year % 100 != 0)
                   || Year % 400 == 0))
      MaxD = 29;                  // năm nhuận
   if (++Day > MaxD)
   {  Day = 1;
      if (++Month > 12) { Month = 1; Year++; }
   }
   return *this;
}
```

Ví dụ 8.3 – Chạy thử

```
Date d1(28, 2, 2024), d2(31, 12, 2025), d3;
cout << d1 << " -> ";  cout << ++d1 << endl;
cout << d2 << " -> ";  cout << ++d2 << endl;
cin >> d3;                        // nhập: 05/10/2026
cout << "Nhap: " << d3 << endl;
```

```
28/02/2024 -> 29/02/2024
31/12/2025 -> 01/01/2026
Nhap: 05/10/2026
```

- **Phân tích:**
  - 2024 là năm nhuận ⇒ sau 28/02 là **29/02**; 31/12/2025 ⇒ sang năm mới **01/01/2026**.
  - `setw(2)` + `setfill('0')` in ngày, tháng đủ 2 chữ số; trả lại `setfill(' ')` để **không ảnh hưởng** các lần xuất sau.
  - Viết `cout << d1 << " -> "; cout << ++d1;` thành **hai** câu lệnh: trong một biểu thức có cả `d1` và `++d1`, các chuẩn C++ cũ **không bảo đảm** thứ tự tính.
- **Cải tiến `>>` cho thực tế:** kiểm tra ngày hợp lệ, nếu sai thì đặt trạng thái lỗi cho dòng nhập:

```
istream &operator>>(istream &is, Date &d)
{  int dd, mm, yy;  char c1, c2;
   is >> dd >> c1 >> mm >> c2 >> yy;
   if (c1 == '/' && c2 == '/' && HopLe(dd, mm, yy))
   {  d.Day = dd;  d.Month = mm;  d.Year = yy;  }
   else
      is.setstate(ios::failbit);   // báo lỗi nhập
   return is;
}
if (!(cin >> d)) cout << "Ngay khong hop le";
```

3.1 Vấn đề sao chép nông với lớp có con trỏ

- Buổi 5: phép gán mặc định **sao chép từng thành viên** ⇒ với lớp có thành viên con trỏ chỉ sao chép **địa chỉ** ("sao chép nông" – shallow copy).

```
class String
{  public:
      String(const char *s = "")
      {  Len = strlen(s);
         Str = new char[Len + 1];
         strcpy(Str, s);
      }
      ~String() { delete[] Str; }
   private:
      char *Str;  int Len;
};
String a("happy"), b("xyz");
b = a;          // b.Str = a.Str  (sao chép NÔNG)
String c = a;   // c.Str = a.Str  (copy constructor mặc định)
```

```
a.Str ─┐
b.Str ─┼──► "happy"      "xyz" ◄── không ai trỏ tới
c.Str ─┘
```

- **Hậu quả:**
  - Sửa chuỗi qua b làm a, c **thay đổi theo**.
  - Vùng "xyz" của b bị **rò rỉ**.
  - Kết thúc chương trình: 3 destructor cùng `delete[]` **một** vùng ⇒ **double free** ⇒ crash (AddressSanitizer báo `attempting double-free` – buổi 5).
- **Sao chép nông xảy ra ở đâu?**

Tình huống	Hàm được dùng
`b = a;` (cả hai đã tồn tại)	**operator=**
`String c = a;` / `String c(a);`	**Copy constructor**
Truyền đối tượng **theo giá trị** vào hàm	Copy constructor
Hàm **trả về** đối tượng theo giá trị	Copy constructor (có thể được tối ưu bỏ qua)

- **Giải pháp – sao chép sâu (deep copy):** cấp phát vùng nhớ **mới** và chép **nội dung** ⇒ tự viết **copy constructor** và **operator=**.

3.2 Copy constructor và operator= cho lớp String

- **Copy constructor:** constructor nhận tham số `const String &` – tạo đối tượng **mới** từ đối tượng đã có.

```
String::String(const String &c)
{  Len = c.Len;
   Str = new char[Len + 1];   // vùng nhớ RIÊNG
   strcpy(Str, c.Str);         // chép nội dung
}
```

- Tham số **phải** là tham chiếu: nếu `String(String c)` thì việc truyền c lại cần gọi copy constructor ⇒ **đệ quy vô hạn** (trình biên dịch báo lỗi).

- **operator=:** gán cho đối tượng **đã tồn tại** (đã có vùng nhớ cũ) ⇒ 4 bước:

```
String &String::operator=(const String &c)
{  if (this != &c)              // 1. kiểm tra tự gán
   {  delete[] Str;              // 2. giải phóng vùng cũ
      Len = c.Len;
      Str = new char[Len + 1];  // 3. cấp phát vùng mới
      strcpy(Str, c.Str);       //    và chép nội dung
   }
   return *this;                // 4. trả về *this
}
```

- **Vì sao kiểm tra tự gán?** Với `s = s;` nếu không kiểm tra: bước 2 xóa Str, bước 3 chép từ `c.Str` – **chính vùng vừa xóa** ⇒ dữ liệu rác.
- **Vì sao trả về `String &`?** Để gán nối tiếp `a = b = c;` (kết hợp phải) như kiểu cơ sở.

Quy tắc ba (Rule of Three)

Nếu một lớp cần tự viết **một** trong ba hàm: **destructor**, **copy constructor**, **operator=** thì gần như chắc chắn cần viết **cả ba** – dấu hiệu là lớp tự quản lý tài nguyên (con trỏ tới vùng nhớ cấp phát động).

3.3 Lớp String hoàn chỉnh (theo giáo trình) – phần khai báo

```
class String
{  friend ostream &operator<<(ostream &, const String &);
   friend istream &operator>>(istream &, String &);
   public:
      String(const char *s = "");          // chuyển kiểu
      String(const String &c);             // copy
constructor
      ~String();                           // destructor
      String &operator=(const String &c);  // gán sâu
      String &operator+=(const String &c); // nối chuỗi
      bool operator==(const String &c) const;
      bool operator<(const String &c) const;
      char &operator[](int i);             // kiểm tra chỉ số
      String operator()(int Index, int SubLen) const;
      int Length() const { return Len; }
   private:
      char *Str;
      int Len;
};
```

```
String &String::operator+=(const String &c)
{  char *t = new char[Len + c.Len + 1];
   strcpy(t, Str);  strcat(t, c.Str);
   delete[] Str;
   Str = t;  Len += c.Len;
   return *this;
}
String String::operator()(int Index, int SubLen) const
{  if (Index < 0 || Index >= Len || SubLen < 0)
      return String("");
   if (Index + SubLen > Len) SubLen = Len - Index;
   char *t = new char[SubLen + 1];
   strncpy(t, Str + Index, SubLen);  t[SubLen] = '\\0';
   String kq(t);
   delete[] t;
   return kq;
}
bool String::operator==(const String &c) const
{  return strcmp(Str, c.Str) == 0;  }
ostream &operator<<(ostream &os, const String &s)
{  os << s.Str;  return os;  }
```

- Lớp String tổng hợp **mọi** kỹ thuật của buổi 7, 8: constructor chuyển kiểu, `+=`, `==`, `<`, `[]`, `()`, `<<`, `>>`, copy constructor, `=`, destructor.

3.4 Kiểm thử sao chép sâu – theo dõi các hàm được gọi

- Thêm lệnh in vào constructor (`[ctor]`), copy constructor (`[copy ctor]`), operator= (`[operator=]`), destructor (`[dtor]`); biên dịch với `-fsanitize=address`.

```
int main()
{  cout << "1) Khoi tao\n";
   String s1("happy"), s2(" birthday"), s3;
   cout << "2) Gan va noi\n";
   s3 = s1;  s3 += s2;
   cout << "s3 = " << s3 << ", s1 = " << s1 << endl;
   cout << "3) Copy constructor\n";
   String s4 = s3;
   s4[0] = 'H';
   cout << "s4 = " << s4 << ", s3 = " << s3 << endl;
   cout << "4) Chuoi con\n";
   String s5 = s3(6, 8);
   cout << "s5 = " << s5 << endl;
   cout << "5) Tu gan\n";
   s4 = s4;
   cout << "s4 = " << s4 << endl;
   cout << "6) Ket thuc\n";
}
```

```
1) Khoi tao
  [ctor "happy"]
  [ctor " birthday"]
  [ctor ""]
2) Gan va noi
  [operator=]
s3 = happy birthday, s1 = happy
3) Copy constructor
  [copy ctor "happy birthday"]
s4 = Happy birthday, s3 = happy birthday
4) Chuoi con
  [ctor "birthday"]
  [copy ctor "birthday"]
  [dtor "birthday"]
s5 = birthday
5) Tu gan
  [operator=]
s4 = Happy birthday
6) Ket thuc
  [dtor "birthday"]
  [dtor "Happy birthday"]
  [dtor "happy birthday"]
  [dtor " birthday"]
  [dtor "happy"]
```

3.5 Phân tích kết quả kiểm thử

- **Bước 2:** `s3 = s1` gọi **operator=** (s3 đã tồn tại); sau `s3 += s2`, s1 vẫn là "happy" ⇒ hai đối tượng **độc lập**.
- **Bước 3:** `String s4 = s3;` – dù có dấu `=`, đây là **khởi tạo** đối tượng mới ⇒ gọi **copy constructor**, không phải operator=.
  - Sửa `s4[0] = 'H'` chỉ đổi s4 ("Happy birthday"); s3 giữ nguyên ⇒ **sao chép sâu hoạt động đúng**.
- **Bước 4:** `s3(6, 8)` tạo `kq` ("birthday") trong hàm, trả về theo giá trị ⇒ **copy constructor** tạo s5, rồi **destructor** hủy kq cục bộ. (Trình biên dịch có thể **tối ưu bỏ** bước sao chép này – RVO.)
- **Bước 5:** `s4 = s4` – nhờ kiểm tra `this != &c`, chuỗi **không bị hỏng**.
- **Bước 6:** 5 destructor theo thứ tự **ngược** thứ tự tạo (s5, s4, s3, s2, s1); AddressSanitizer **không** báo lỗi ⇒ không rò rỉ, không giải phóng hai lần.

Ví dụ 8.4 – Thử bỏ copy constructor

Xóa copy constructor (dùng bản mặc định) và chạy lại: `s4[0] = 'H'` làm **s3 cũng thành** "Happy birthday", cuối chương trình AddressSanitizer báo **attempting double-free** ⇒ chứng minh sự cần thiết của sao chép sâu.

Câu lệnh	Hàm được gọi
`String a("x");`	Constructor
`String b = a;` / `String b(a);`	Copy constructor
`b = a;`	operator=
`F(a);` với `void F(String s)`	Copy constructor
`F(a);` với `void F(const String &s)`	**Không** gọi gì (không sao chép)
`return kq;` (trả giá trị)	Copy constructor (hoặc được tối ưu)

3.6 Áp dụng Quy tắc ba: lớp Mang

```
class Mang
{  friend ostream &operator<<(ostream &os, const Mang &m);
   public:
      explicit Mang(int n = 0)
         : N(n), A(n > 0 ? new int[n] : NULL)
      {  for (int i = 0; i < N; i++) A[i] = 0;  }
      Mang(const Mang &m)                    // 1. copy ctor
         : N(m.N), A(m.N > 0 ? new int[m.N] : NULL)
      {  for (int i = 0; i < N; i++) A[i] = m.A[i];  }
      ~Mang() { delete[] A; }                // 2. destructor
      Mang &operator=(const Mang &m)         // 3. gán
      {  if (this != &m)
         {  int *B = m.N > 0 ? new int[m.N] : NULL;
            for (int i = 0; i < m.N; i++) B[i] = m.A[i];
            delete[] A;  A = B;  N = m.N;
         }
         return *this;
      }
      int &operator[](int i) { return A[i]; }
      int Size() const { return N; }
      friend Mang operator+(const Mang &x, const Mang &y);
   private:
      int N;  int *A;
};
```

```
Mang TaoMang(int n)
{  Mang m(n);
   for (int i = 0; i < n; i++) m[i] = i * i;
   return m;                          // trả về theo giá trị
}
int main()
{  Mang a = TaoMang(4), b(2);
   b[0] = 7;  b[1] = 8;
   Mang c = a + b;                    // nối hai mảng
   Mang d;  d = c;  d[0] = 99;
   cout << a << "\n" << c << "\n" << d << endl;
   a = a;  cout << a.Size() << endl;
}
```

```
[0, 1, 4, 9]
[0, 1, 4, 9, 7, 8]
[99, 1, 4, 9, 7, 8]
4
```

- d sửa phần tử đầu nhưng c **không đổi** ⇒ sao chép sâu đúng; AddressSanitizer không báo lỗi.
- **Kỹ thuật an toàn trong operator=:** cấp phát vùng mới B **trước**, chép xong rồi mới `delete[] A` ⇒ nếu `new` thất bại, đối tượng cũ vẫn nguyên vẹn; đồng thời tự xử lý được tự gán.

4.1 Tóm tắt buổi 8

Toán tử	Dạng cài đặt	Ghi nhớ
`[]`	Thành viên, trả `T &` (+ bản const)	Kiểm tra chỉ số; dùng được hai vế phép gán
`()`	Thành viên, số tham số tùy ý	Ma trận a(i, j), chuỗi con, đối tượng hàm
`operator double()`	Thành viên, không kiểu trả về	Chuyển lớp ⇒ kiểu khác; coi chừng nhập nhằng
`new`, `delete`	Thành viên (static ngầm) hoặc toàn cục	Chỉ thay cấp phát bộ nhớ
`<<`, `>>`	**friend**, trả `ostream &` / `istream &`	Dùng tham số os/is để chạy được với file
Copy constructor	`T(const T &)`	Khởi tạo từ đối tượng khác; tham số phải là tham chiếu
`operator=`	`T &operator=(const T &)`	Kiểm tra tự gán, giải phóng cũ, cấp mới, `return *this`

Câu hỏi nhanh

- `String s = "abc";` gọi những hàm nào? → **Constructor** `String(const char *)` (khởi tạo, không phải gán).
- Lớp Time (3 int, không con trỏ) có cần tự viết copy constructor không? → **Không** – sao chép mặc định đã đúng.

4.2 Bài thực hành buổi 8 – Lớp String có cấp phát động, kiểm thử sao chép sâu

Yêu cầu

- 1) Cài đặt lớp **String** như mục 3.3: constructor từ `const char *`, copy constructor, destructor, `=`, `+=`, `+` (friend, trả về String mới), `==`, `!=`, `<`, `[]` (hai phiên bản), `()` chuỗi con, `<<`, `>>`, `Length()`.
- 2) Thêm lệnh in vào constructor/copy constructor/operator=/destructor; chạy chương trình kiểm thử mục 3.4 và **giải thích từng dòng** kết quả.
- 3) Kiểm thử **bắt buộc** (biên dịch `-fsanitize=address`): tự gán `s = s`; gán nối tiếp `a = b = c`; truyền String theo giá trị vào hàm; trả về String từ hàm; mảng 3 String; String rỗng.
- 4) Lần lượt xóa copy constructor, rồi xóa operator= (giữ bản mặc định); ghi lại lỗi mà AddressSanitizer báo và giải thích.
- 5) Viết lớp **Mang** (mảng số nguyên động) theo Quy tắc ba: `[]` kiểm tra chỉ số, `<<` in mảng, `==`, `+` nối hai mảng.

4.3 Câu hỏi và bài tập

Câu hỏi

- 1) Vì sao `operator[]` thường trả về tham chiếu? Vì sao cần hai phiên bản const và không const?
- 2) Vì sao `operator<<` không thể là hàm thành viên của lớp Date? Vì sao phải trả về `ostream &`?
- 3) Sao chép nông là gì? Xảy ra ở những tình huống nào? Nêu "Quy tắc ba".
- 4) Phân biệt copy constructor và operator=. Vì sao operator= phải kiểm tra tự gán?

Bài tập dự đoán (có đáp số)

Với lớp String có in thông báo như mục 3.4, đoạn mã sau gọi những hàm nào?

```
void In(String s) { cout << s; }
String a("hi"), b;
b = a;
In(a);
String c(b);
```

→ **Đáp số:** `[ctor "hi"]`, `[ctor ""]` (b), `[operator=]`, `[copy ctor "hi"]` (tham số s) rồi in `hi` và `[dtor "hi"]` (s hủy khi In kết thúc), `[copy ctor "hi"]` (c).

Bài tập lập trình

- 1) Lớp **DaThuc** (hệ số lưu mảng động): `+`, `*`, `()` tính giá trị tại x, `[]` truy cập hệ số, `<<`; theo Quy tắc ba.
- 2) Lớp **MaTran** m × n: `()` truy cập phần tử, `+`, `*` (kiểm tra kích thước), `<<`, `>>`, copy constructor, `=`.